DARKHOLE: 1

Capture-the-Flag Security Assessment Report

Target	darkhole.lan
Victim IP	192.168.31.113
Assessment type	Authorized local CTF / lab environment
Scope	Services and application paths evidenced in the supplied screenshots
Report date	14 October 2026

Confidentiality / redaction note

This report intentionally retains only the victim hostname and victim IP. Other discovered hostnames/IP addresses and personal identifiers are omitted. Passwords, session tokens, and other secret values are not reproduced. Any lab credentials visible in the evidence are also withheld.

Prepared from user-supplied screenshots and the provided workbook. Findings are limited to what those materials demonstrate; unverified issues are labelled accordingly.

1. Executive summary

The supplied evidence shows a web application on TCP/80 and SSH on TCP/22. The application exposes several PHP endpoints and an upload function. The demonstrated attack path appears to progress from web upload handling to a shell running as **www-data**, then to the local user **john** through a PATH-resolution issue, and finally to root through a sudo-permitted Python script that was modified to execute a shell.

The screenshots also show a password-change request containing an object identifier, directory enumeration results, a server-side upload restriction message, a SUID enumeration command, and a root shell. The report distinguishes observed evidence from risks that need further validation.

Overall impact: **Critical in the lab context** because the evidence shows root-level compromise. This rating describes demonstrated impact, not a CVSS score.

2. Scope and methodology

In-scope target: darkhole.lan (192.168.31.113). Reconnaissance used Nmap host discovery and service detection, followed by Gobuster directory enumeration and manual web request testing. Post-exploitation screenshots show shell identity checks, local file/permission inspection, PATH manipulation, and sudo rule inspection.

No claim is made that every possible service or vulnerability was tested. Only the ports and issues visible in the screenshots are documented below.

3. Port and service inventory

Port	Observed service	Evidence / notes	Assessment
22/TCP	SSH — OpenSSH 8.2p1 Ubuntu 4ubuntu0.2	Nmap service detection; later SSH login shown in evidence.	Service exposure / credential access path; no SSH-specific CVE demonstrated.
80/TCP	HTTP — Apache httpd 2.4.41 (Ubuntu)	Nmap banner/title; web application and PHP endpoints enumerated.	Multiple application-level weaknesses observed.

The supplied screenshots do not show confirmed open TCP ports other than 22 and 80 for the target. Do not infer that ports not listed are vulnerable or exposed.

4. Findings and proof of concept

F-01 — Web upload validation bypass leading to server-side code execution

Port: 80/TCP **Severity:** Critical (demonstrated lab impact) **Status:** Exploitation shown

Evidence: The upload page displays an allow-list message for JPG/PNG/GIF. A screenshot shows a file renamed from `reverse.png` to `reverse.phtml`. A subsequent Netcat listener receives a shell whose identity is `uid=33(www-data)`.

Why it matters: If the server accepts an executable PHP-compatible extension or executes uploaded content from a web-accessible directory, an attacker may run commands as the web-server account.

POC — evidence-based reproduction in the isolated CTF only:

```
1. Enumerate the web paths and inspect the upload form.
2. Submit a harmless test image and observe server-side validation.
3. In the lab, test whether extension/MIME validation can be bypassed; the screenshot records a .phtml
rename attempt.
4. The supplied evidence then shows a listener on the tester side and a received shell as www-data.
5. Verify access level with: id; whoami
```

Remediation: Store uploads outside the web root; generate server-side filenames; validate actual file content and extension using a strict allow-list; reject PHP/PHTML and double extensions; disable script execution in upload directories; enforce least privilege; and log/review upload attempts.

F-02 — Password-change endpoint may allow object-level authorization bypass (IDOR)

Port: 80/TCP **Severity:** High (needs controlled validation) **Status:** Suspicious request shown

Evidence: Burp shows a POST to `/dashboard.php?id=4` with body fields `password=…&id=1`. The response displays “Password Has been Updated”. This suggests the body ID may control which account is changed independently of the URL ID, but the screenshot alone does not prove unauthorized cross-account modification.

POC: In an authorized test account, compare the URL ID and body ID while changing only your own test password. Verify which account is affected. Do not test against real users or reuse personal credentials.

```
POST /dashboard.php?id=4
Content-Type: application/x-www-form-urlencoded

password=[REDACTED]&id=1
```

Remediation: Derive the account identity from the authenticated server-side session, enforce object-level authorization for every change, ignore client-supplied account IDs for sensitive actions, use CSRF protection, and audit password-change events.

F-03 — Local privilege escalation via PATH hijacking

Location: Local host after initial shell **Severity:** High **Status:** Demonstrated as user john

Evidence: The screenshots show a file named `toto` that executes `id` by name rather than an absolute path. A writable temporary directory is added early in PATH, and execution output changes to `uid=1001(john)`. This indicates a privileged or different-user execution context resolving a command through attacker-controlled PATH.

POC — safe validation summary:

```
1. Inspect the script/binary and confirm it invokes a command without an absolute path.
2. Check ownership, write permissions, and the execution identity.
3. In the disposable CTF, compare command resolution before and after a controlled PATH change.
4. The screenshot records the resulting identity as john.
```

Remediation: Use absolute paths for external commands, sanitize PATH in privileged programs, avoid executing writable scripts/binaries, set secure ownership/permissions, and remove unnecessary SUID/privileged execution.

F-04 — Unsafe sudo permission for a user-writable Python script

Location: Local host **Severity:** Critical **Status:** Root escalation shown

Evidence: The SSH session shows sudo permitting `/usr/bin/python3 /home/john/file.py` as root without a password. The later screenshot shows the script changed to import `os` and invoke a shell, after which `id` reports UID/GID 0 and the prompt becomes root.

POC: Review the sudo policy and the script's ownership/write permissions. The supplied screenshots demonstrate that the allowed Python script was modified and then executed through sudo, resulting in root. Secret values and credentials are intentionally omitted from this report.

```
sudo -l
ls -l /home/john/file.py
# Evidence outcome shown in screenshot after permitted script execution:
id
whoami
```

Remediation: Remove broad sudo permission for interpreters; do not allow sudo execution of user-writable scripts; make privileged scripts root-owned and non-writable by unprivileged users; restrict allowed arguments and commands; and review sudoers entries.

F-05 — Web directory and endpoint exposure

Port: 80/TCP **Severity:** Medium (context dependent) **Status:** Enumerated

Evidence: Gobuster identified `dashboard.php`, `index.php`, `login.php`, `logout.php`, `register.php`, `upload/`, and configuration/CSS/JS paths. Some dotfiles and server-status paths returned 403. A 403 response does not itself prove that sensitive contents were disclosed.

POC:

```
gobuster dir -u http://192.168.31.113 -w /usr/share/wordlists/dirb/big.txt -x php
```

Remediation: Remove unused files and endpoints, disable directory listing, ensure configuration files are not web-accessible, restrict server-status to administrators, and enforce authentication/authorization server-side.

F-06 — SSH service exposed to the lab network

Port: 22/TCP **Severity:** Low to Medium (exposure finding) **Status:** Confirmed open

Evidence: Nmap reports OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 on TCP/22. A later screenshot shows a successful SSH session as john. The evidence does not demonstrate an SSH protocol vulnerability; access appears to rely on credentials obtained in the lab.

POC:

```
nmap -sC -sV -p 22 192.168.31.113
ssh john@192.168.31.113
```

Remediation: Restrict SSH to trusted management hosts, use key-based authentication where possible, disable password login when operationally feasible, apply security updates, and monitor authentication failures.

5. Local enumeration evidence

The screenshots show enumeration of SUID files using `find / -type f -perm -4000 2>/dev/null`, discovery of files in `/home/john`, a denied read of `user.txt` before privilege escalation, and a sudo rule for a Python script. These are supporting evidence for the PATH-hijack and sudo misconfiguration findings; the report does not reproduce any flag or password.

```
find / -type f -perm -4000 2>/dev/null
sudo -l
ls -la /home/john
```

The provided screenshots also show a password file and account credentials. Those values have deliberately been excluded from this report and README to avoid disclosing secrets.

6. Remediation priority

Priority	Action
P1 — Immediate	Remove the sudo-to-interpreter path; make privileged scripts root-owned/non-writable; disable execution of uploaded files; fix upload validation.
P2 — High	Enforce account-level authorization on password changes; derive target account from the authenticated session; add CSRF controls and audit logging.
P3 — Medium	Reduce exposed web paths, remove unused endpoints, restrict server-status and SSH, and update Apache/OpenSSH packages.
P4 — Verify	Retest each finding after remediation; preserve logs; rotate any lab credentials that were reused outside the disposable environment.

7. Limitations

This report is based on screenshots, not a fresh scan or direct access to the target. HTTP request/response behavior, upload execution controls, and authorization impact should be revalidated in the isolated lab. The supplied workbook appears to describe FTP/vsftpd on port 21, which is not supported by the DarkHole 1 screenshots; it was therefore not used as evidence for this target. No FTP finding is included.

8. Conclusion

The supplied evidence demonstrates a lab compromise chain from a web upload path to a www-data shell, then to john, and ultimately root through an unsafe sudo rule allowing execution of a modifiable Python script. Fixing the upload execution behavior and privileged-script permissions should be the highest priorities.

Assessment scope: darkhole.lan (192.168.31.113). Other discovered hostnames/IP addresses and credential values have been excluded.